Project Proposal: Self-Play Reinforcement Learning on Othello – AlphaZero vs. PPO

Mohammed Sanan Moinuddin

CS 5180: Reinforcement Learning – Spring 2026

1 Problem Description

I study self-play reinforcement learning in **Othello** (Reversi), a two-player zero-sum board game on an 8×8 grid. Players alternate placing discs; opponent discs flanked between the new disc and an existing friendly disc are flipped. The player with the most discs when no legal moves remain wins.

Othello is a compelling RL domain because it is (1) a *zero-sum, perfect-information* game with a clean reward signal, (2) *not a solved game* (unlike Connect4), so there is no artificial ceiling on agent performance, (3) fully deterministic, making it straightforward to attribute outcomes to the agent's policy, and (4) requires *long-horizon planning* (up to 60 moves), providing a serious test for RL.

Formal MDP formulation: Othello is technically a two-player Markov game, but by encoding the board from the current player's perspective, I reduce it to a single-agent MDP where the opponent is part of the environment.

- **State** s: Tensor of shape $(3, 8, 8)$–channel 0 for current-player discs, channel 1 for opponent discs, channel 2 as a turn indicator. This canonical representation lets a single network play as either color.
- **Action** a: Integer in $\{0, \ldots, 63\}$ for disc placement. Illegal moves are masked before the policy softmax.
- **Reward** r: Terminal only – $+1$ (win), -1 (loss), 0 (draw).
- **Transition**: Deterministic, governed by Othello rules.
- **Horizon**: At most 60 moves (64 squares minus 4 pre-placed).

Environment: I use the Othello implementation from the open-source `alpha-zero-general` framework, which provides board logic, legal-action queries, and terminal detection in Python. Both algorithms share this single game implementation to ensure identical game semantics. Prior RL work on Othello includes TD-learning with self-play [1], CNN-based move prediction that defeated the Edax engine [2], and the recent MiniZero framework comparing AlphaZero and MuZero on 8×8 Othello [3]. However, MiniZero compares two MCTS-based algorithms; my contribution is a comparison between a *search-based* method (AlphaZero) and a *search-free* policy gradient method (PPO), which has not been studied on Othello.

2 Algorithms

I will be implementing both algorithms: for AlphaZero, I adapt the `alpha-zero-general` framework[1], tuning the neural network and self-play training loop for 8×8 Othello; for PPO, I implement the full algorithm from scratch. Shared infrastructure–baselines, evaluation, and Elo computation–is written once and used by both.

Algorithm 1: AlphaZero: AlphaZero [4] combines a CNN with Monte Carlo Tree Search (MCTS). The CNN takes the $(3, 8, 8)$ board and outputs a *policy* $\mathbf{p}(s)$ (distribution over actions) and a *value* $v(s) \in [-1, +1]$. During play, MCTS runs N simulations using the PUCT selection rule:

$$\mathrm{PUCT}(s, a) = Q(s, a) + c_{\mathrm{puct}} \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

where $Q(s, a)$ is the mean value of prior visits and $P(s, a)$ is the network's prior. Training proceeds via self-play: games produce tuples $(s, \boldsymbol{\pi}_{\mathrm{mcts}}, z)$ stored in a replay buffer, and the network minimizes $\mathcal{L} = (v(s) - z)^2 - \boldsymbol{\pi}_{\mathrm{mcts}}^{\top} \log \mathbf{p}(s)$.

[1] `https://github.com/suragnair/alpha-zero-general`

Algorithm 2: PPO with Opponent-Pool Self-Play: Proximal Policy Optimization [5] directly optimizes a stochastic policy via the clipped surrogate objective:

$$\mathcal{L}_{\text{CLIP}} = \mathbb{E}\Big[\min\Big(r(\theta)\hat{A},\ \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}\Big)\Big]$$

where $r(\theta) = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$ and $\hat{A}$ is the generalized advantage estimate. The clipping prevents destabilizing large policy updates. I use an actor-critic CNN with the same architecture as AlphaZero for fair comparison. For self-play, I maintain an *opponent pool*: every K steps the current policy is checkpointed, and opponents are sampled uniformly from the pool to prevent catastrophic forgetting, following OpenAI Five [6].

Key difference: AlphaZero combines *learning with planning* (MCTS lookahead), while PPO relies on *pure learned intuition* with no search at decision time.

Baselines.

- **Random agent**: selects uniformly among legal moves (lower bound).
- **Minimax with alpha-beta pruning** at depth 4 and depth 6, using a positional heuristic (corners and edges weighted heavily).
- **Head-to-head**: AlphaZero vs. PPO directly.

3 Expected Results

I will report: (1) **win rates** (with 95% confidence intervals) from round-robin tournaments of 200+ games per matchup, (2) **Elo ratings** ranking all agents on a single scale, and (3) **training curves** (win rate vs. training iterations against fixed baselines). As a stretch goal, I will include **ablation studies**–varying MCTS simulation count (50/200/800) for AlphaZero and opponent pool size (1/5/20) for PPO. Training will be conducted on Google Cloud (GPU).

Hypothesis: AlphaZero > PPO $\gg$ Minimax (depth 6) > Minimax (depth 4) > Random, since MCTS lookahead should provide an edge over a pure policy network. However, the magnitude of this gap–and whether PPO's faster training compensates at lower compute budgets–is the central open question.

Risks and mitigations:

- *MCTS compute cost*: Google Cloud GPUs; reduced simulation counts during training, increased for evaluation.
- *PPO self-play collapse*: Opponent pool diversity; continuous monitoring against fixed baselines.
- *Slow convergence*: Early start with short sanity-check runs before full training.
- *Inconclusive results*: 200+ games per matchup for statistical significance.

References

[1] Michiel van der Ree and Marco Wiering. Reinforcement learning in the game of Othello: Learning against a fixed opponent and learning from self-play. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 108–115, 2013.

[2] Paweł Liskowski, Wojciech Jaśkowski, and Krzysztof Krawiec. Learning to play Othello with deep neural networks. *IEEE Transactions on Games*, 10(4):354–364, 2018.

[3] Ti-Rong Wu, Hung Guei, Pei-Chiun Peng, Po-Wei Huang, Ting Han Wei, Chung-Chin Shih, and Yun-Jui Tsai. MiniZero: Comparative analysis of AlphaZero and MuZero on Go, Othello, and Atari games. *arXiv preprint arXiv:2310.11305*, 2023.

[4] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[6] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.